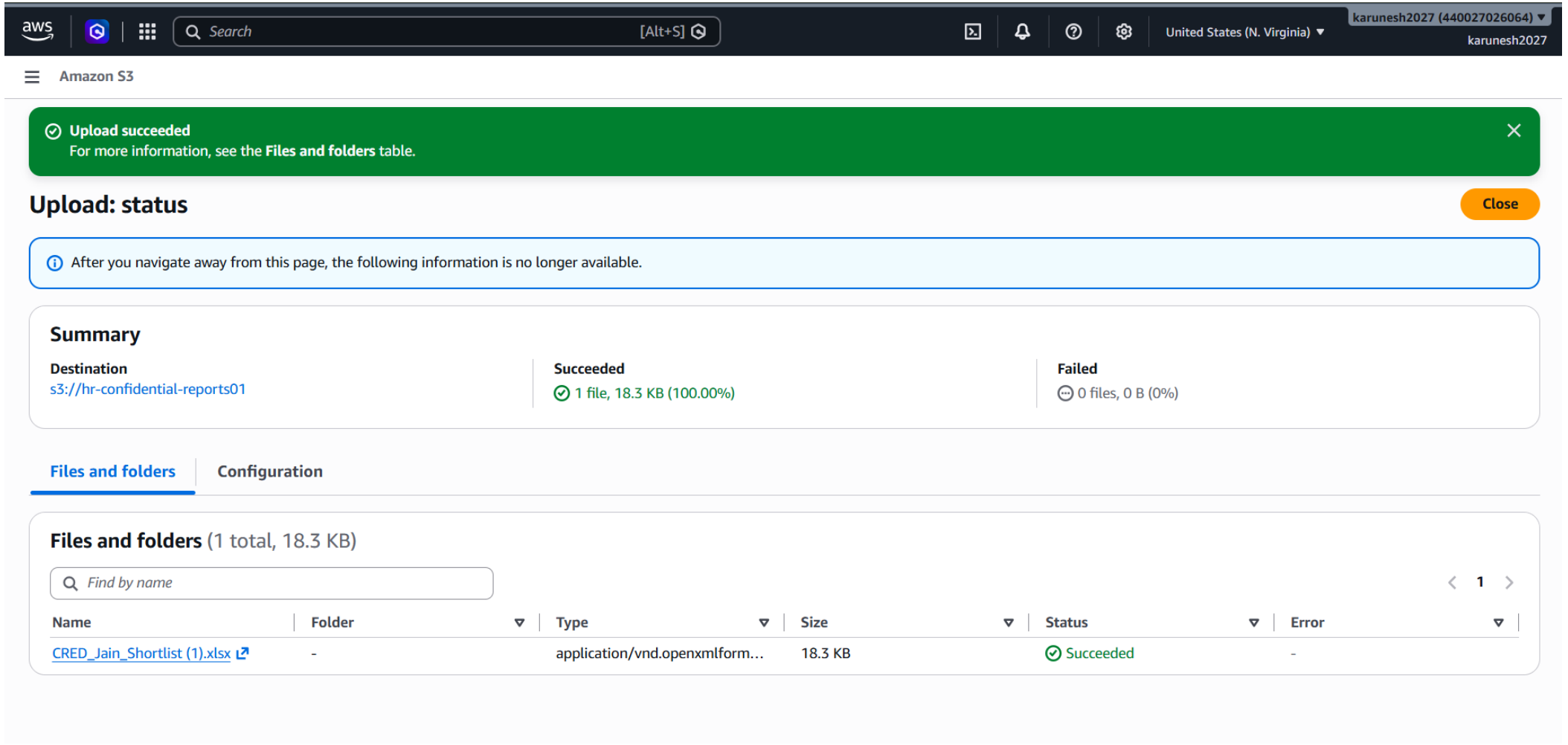
AWS CLOUD SERVICES

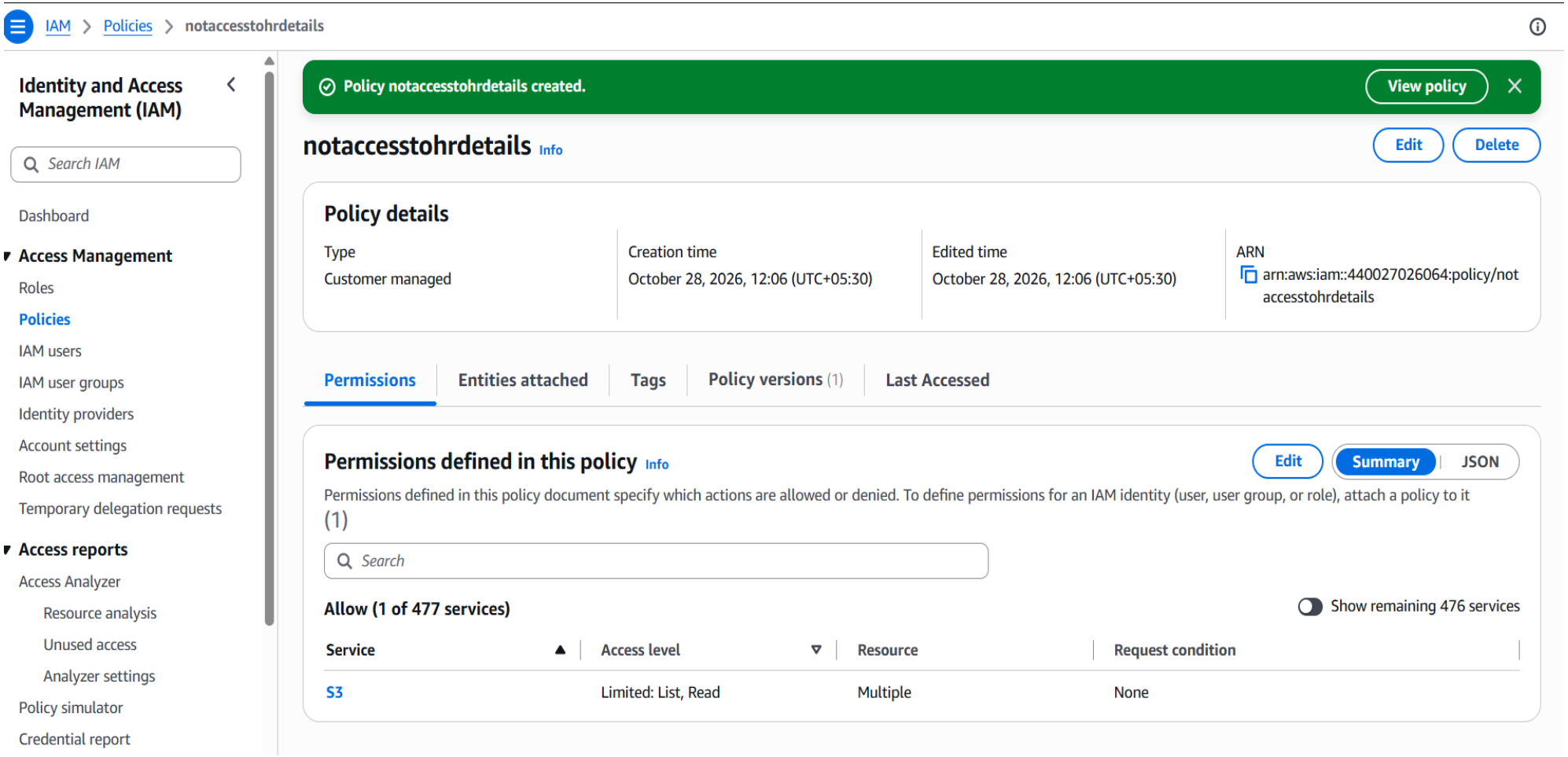
Question 10: S3 and IAM Security



AWS CLOUD SERVICES

Question 10: S3 and IAM Security

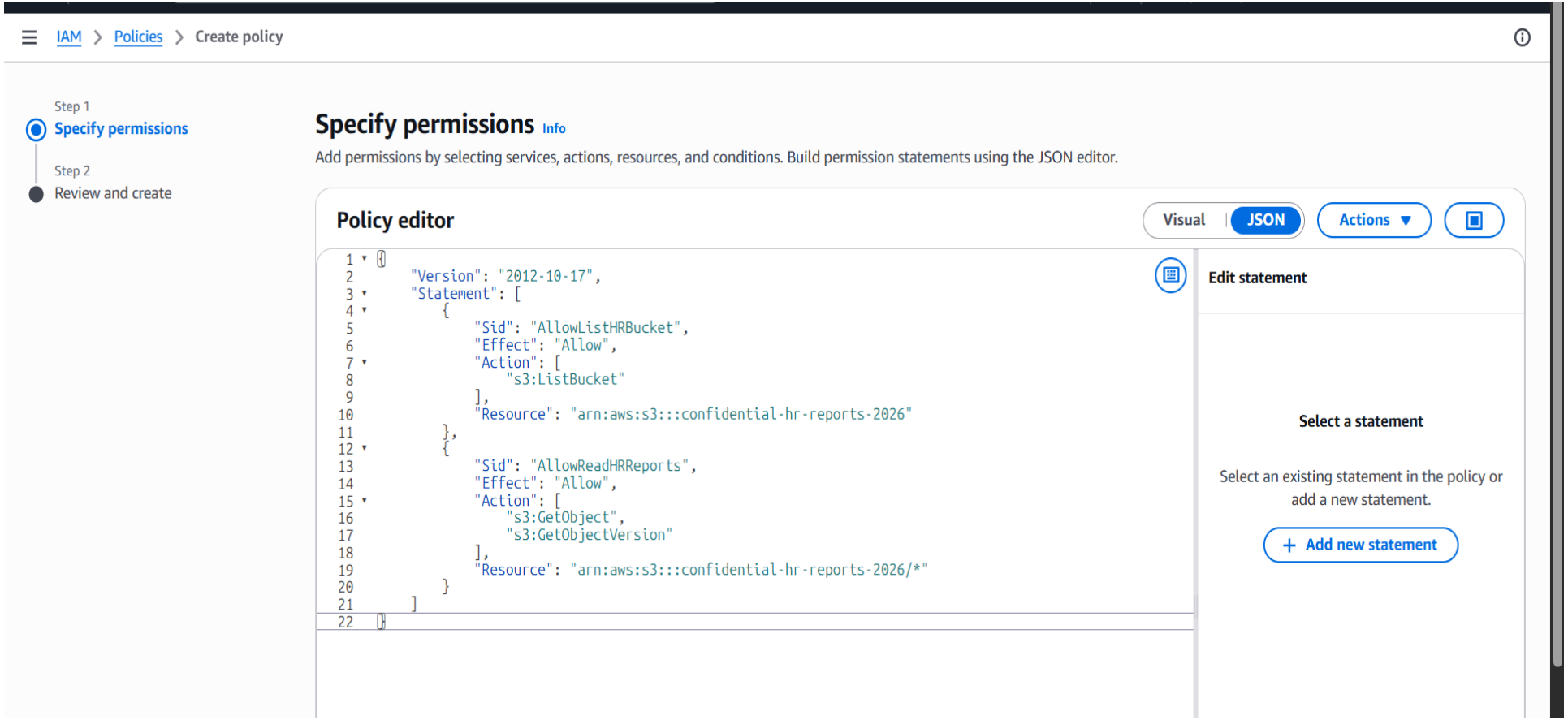
b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 mark



AWS CLOUD SERVICES

Question 10: S3 and IAM Security

b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 mark



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

- To verify that an unauthorized user or public entity cannot read the file, you can perform two verification tests:
- **Simulate via AWS IAM Policy Simulator:**
  - Open the AWS IAM Policy Simulator.
  - Select an unauthorized user or role profile.
  - Test the s3:GetObject action on arn:aws:s3:confidential-hr-reports-2026/employee_report_sample.pdf.
  - The expected result must be **Implicitly Denied** or **Explicitly Denied**.

d. Explain how encryption at rest protects the stored data. (1 mark)

- **Encryption at rest** protects the stored data by ensuring that the underlying physical storage media (the actual hard drives and data centers managed by AWS) cannot be read directly if compromised.
- Even if an attacker physically steals a hard drive from an AWS data center, or bypasses the S3 virtualization layer to access the raw binary storage blocks, the files will appear as unreadable ciphertext.
- The data can only be decrypted when accessed through the authorized S3 software layer, which verifies that the user has valid IAM permissions to use both the S3 bucket *and* the associated encryption key (such as an AWS KMS key).

a. Check the EC2 instance state and status checks. (1 mark)

- To identify if the issue stems from an infrastructure or virtual machine failure, check the instance's state and system diagnostics using the AWS Management Console or the AWS CLI.
- **Using the AWS CLI:**
- bash
- aws ec2 describe-instance-status --instance-ids i-0123456789abcdef0

b. Explain how security groups and route tables can affect application connectivity. (1 mark)

- Network connectivity relies on both structural firewall rules and proper network routing pathways working together.
- **Security Groups (Stateful Firewalls):** These control traffic at the instance level. If the inbound rules do not explicitly permit traffic on the web application's port (e.g., HTTP port 80 or HTTPS port 443) from the client's IP range or the internet (0.0.0.0/0), incoming requests are silently dropped. Because they are **stateful**, if inbound traffic is allowed, return traffic is automatically allowed.
- **Route Tables (Network Pathways):** These direct subnet traffic out to external networks. For users to connect over the public internet, the EC2 instance must reside in a public subnet whose associated route table contains a target entry directing traffic (0.0.0.0/0) out through an **Internet Gateway (IGW)**. Missing or misconfigured routes create a routing black hole, breaking all external connectivity.

C. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

- **AWS Console Step-by-Step Flow:**
- **Select Metric:** Open the CloudWatch console, go to **Alarms** > **Create Alarm**, and select the AWS/EC2 > Per-Instance Metrics -> CPU Utilization metric for your specific instance.
- **Define Conditions:** Set the **Period** to 5 minutes and choose **Static** threshold type. Configure the alarm to trigger when the value is **Greater than** 85.
- **Set Evaluation Period:** Expand **Additional configuration** and configure **Datapoints to alarm** to 2 out of 2 (meaning CPU must average >85% across two consecutive 5-minute intervals before alerting, preventing false alarms from brief spikes).
- **Configure Action:** Under the **Notification** section, set the state trigger to In alarm and choose or create an **Amazon SNS Topic** to email or alert the engineering team.

D. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

- **Highest-Risk Diagnostic Investigation: Operating System Analysis**
- The most effective way to pinpoint the exact root cause is to analyze the internal runtime processes within the guest operating system:
- **Access the Instance:** Establish an active terminal connection using **SSM Session Manager** or secure **SSH**.
- **Execute Diagnostic Tools:** Run interactive system monitors to view live process loads:
  - **Linux:** Execute the `top` or `htop` command.
  - **Windows:** Open **Task Manager** or run `Get-Process | Sort-Object CPU -Descending` in PowerShell.
- **Analyze Results:** Identify the top-consuming process ID (PID). This will reveal whether the slowdown is caused by a runaway application thread, an unoptimized database query, a memory leak swapping to disk, or an external surge in user traffic (such as a DDoS attack visible in web server access logs).

Question 12: Event-Driven Processing (5 Marks)

- **a. Execution Service for Processing Code (1 Mark)**
- **AWS Lambda** is the most suitable service for executing this processing code.
- As a serverless compute service, it automatically runs your code without the need to provision or manage servers.
- It scales precisely with the volume of incoming files, executes within seconds of an upload, and incurs charges only for the exact compute time used per execution, making it highly cost-effective for event-driven file processing.

Question 12 B :

- **b. S3 Event Trigger Mechanism (1 Mark)**
- An S3 upload triggers the processing code through **Amazon S3 Event Notifications**:
- You configure an event notification on the source S3 bucket, specifying the event type as s3:ObjectCreated:Put or s3:ObjectCreated:*.
- To ensure only specific files are processed, you can optionally restrict the notification using a prefix (e.g., uploads/) or a suffix (e.g., .csv).
- When a matching file lands in the bucket, Amazon S3 generates a structured JSON event payload containing metadata about the file (such as bucket name and file key) and sends it directly to the AWS Lambda function, invoking the code instantly.
-

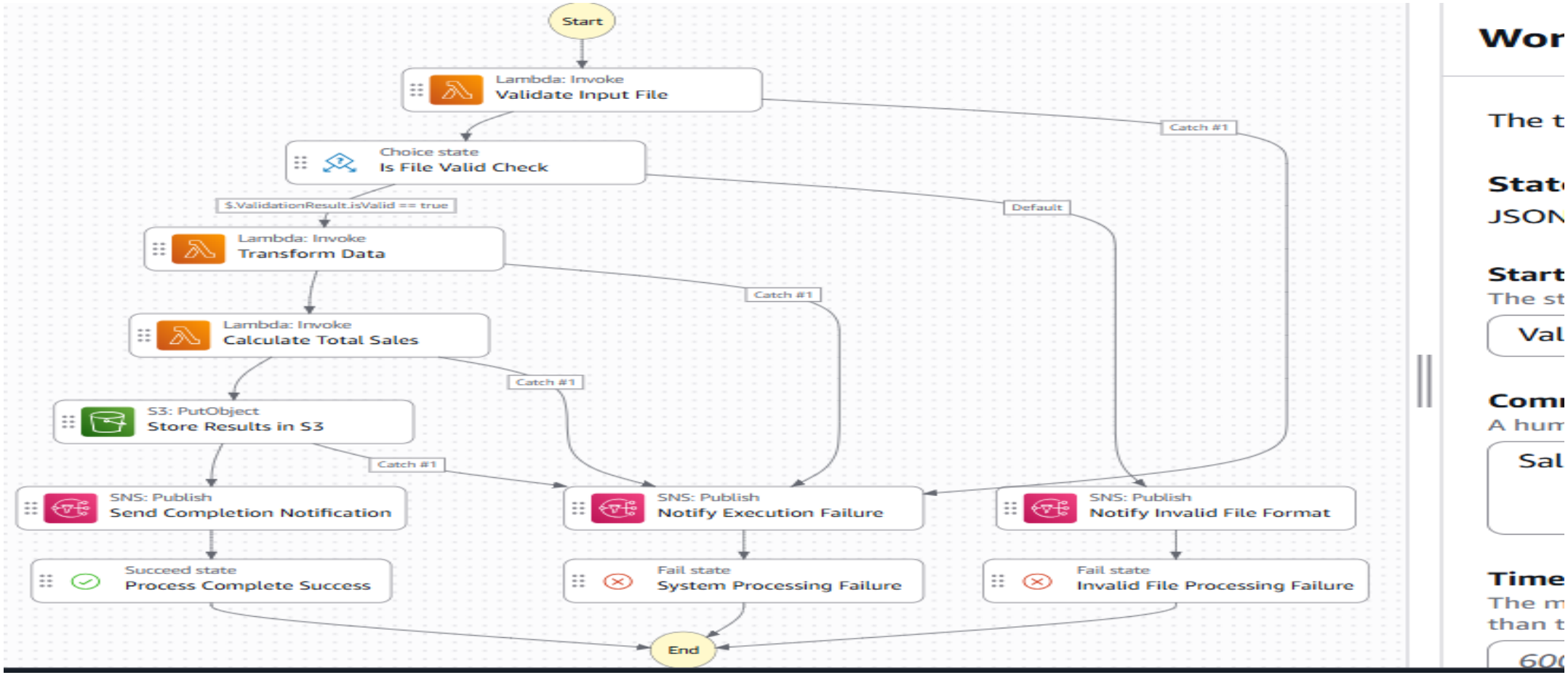
Question 12 C :

- The Lambda function must be associated with an **Execution Role** containing a policy that enforces the principle of least privilege. It requires separate permissions for reading the input data and writing the output data:
- **Read Permissions (Source Bucket):** The policy must grant s3:GetObject on the specific source bucket resource pathway (arn:aws:s3:::source-csv-bucket/*). This allows the code to download and read the uploaded CSV payload.
- **Write Permissions (Destination):** The policy must grant s3:PutObject on the target storage resource pathway. For example, if the destination is another S3 bucket, it needs s3:PutObject on arn:aws:s3:::processed-results-bucket/* to save the validated records.

Question 12 D :

- System logs and run-time execution errors are systematically captured using native monitoring tools:
- **Execution & Application Logs:** AWS Lambda automatically integrates with **Amazon CloudWatch Logs**. Any standard print statements or runtime errors generated by the processing code are captured inside CloudWatch Log Streams mapped to that specific function.
- **Resilience & Dead Letter Queues (DLQ):** To prevent permanent data loss when validation or processing completely fails, the Lambda function can be configured with an **Amazon SQS queue** or an **Amazon SNS topic** as a Dead Letter Queue (DLQ) or On-Failure destination. Corrupted or unprocessable CSV events are routed there for engineering inspection without halting the main pipeline.

SECTION E: AWS STEP FUNCTIONS
Question 13: State Machine Design (7 Marks)



SECTION E: AWS STEP FUNCTIONS
Question 13: State Machine Design (7 Marks)

